Arrays

One-Dimensional Arrays :

The general form of a one dimensional array declaration is

```
type var-name[];
```

Example:

```
int month_days[];
month_days = new int[12];
```

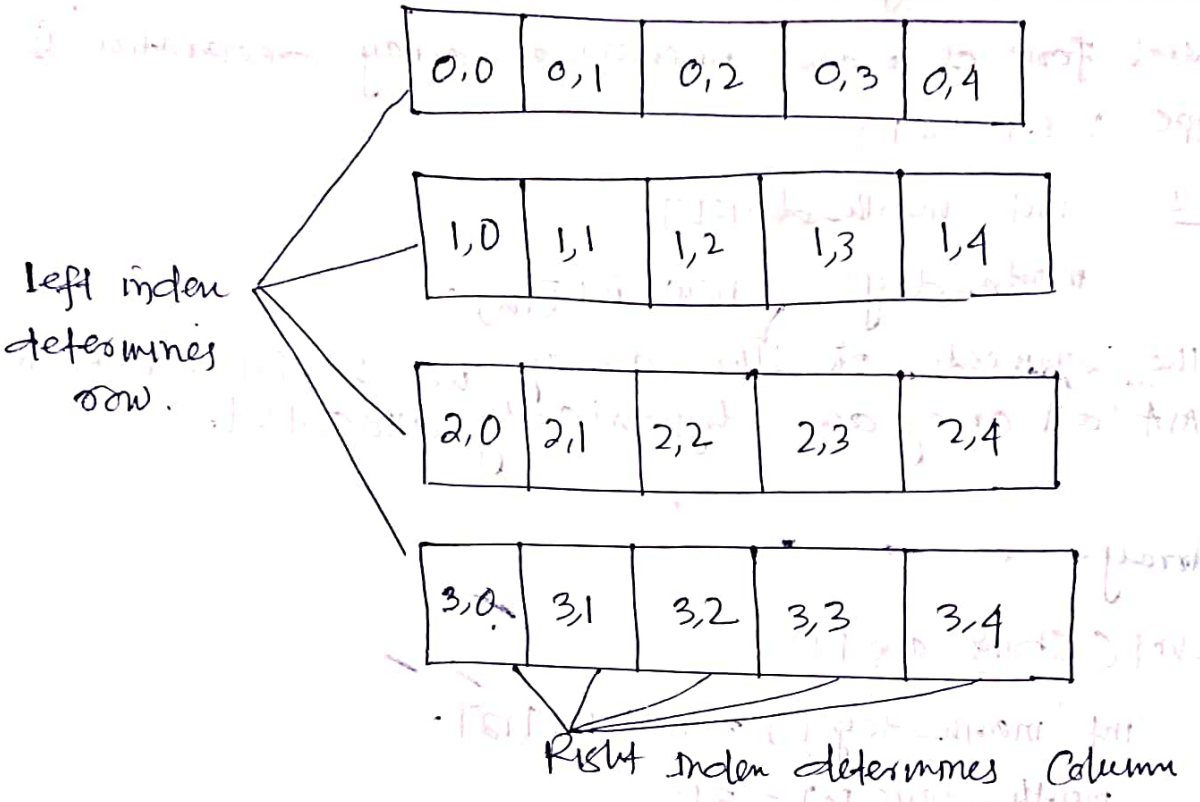
All the elements in the array will be initialized to 0.
In JAVA all array are dynamically allocated.

```
class Array
{
    PSVM ( String args[])
    {   int month_days[] = new int[12];
        month_days[0] = 31;
        month_days[1] = 28;
              :
        month_days[10] = 30;
        month_days[11] = 31;
        SOP (" December has " + month_days[11] + "days");
    }
}

class AutoArray
{
    PSVM (String args[])
    {
        int month_days[] = { 31,28, 31, 30, 31, 30, 31, 31,
                             30, 31, 30, 31 };
        SOP (" December has" + month_days[3] + " days :");
    }
}
```

Multidimentional Arrays :

```
int twoD[][] = new int[4][5];
```




```
Class TwoArray
{
    PSVM(String args[])
    {
        int twoD[] = new int[4][5];
        int i, j, K = 0;

        for(i=0; i<4; i++)
            for(j=0; j<5; j++)
            {
                twoD[i][j] = K;
                K++;
            }
        for(i=0; i<4; i++)
        {   for(j=0; j<5; j++)
                SOP(twoD[i][j] + " ");
            SOP();
        }
    }
}
```

```
class TwoDArray
{
    PSVM ( String args[])
    {
        int twoD [][] = new int [4] [];
        twoD[0] = new int [1];
        twoD [1] = new int [2];
        twoD [2] = new int [3];
        twoD [3] = new int [4];
        int i, j, k = 0;
        for ( i=0; i<4; i++)
            for ( j=0; j< i+1; j++)
            {
                twoD[i][j] = k;
                k++;
            }
        for ( i=0; i<4; i++)
        {   for (j=0; j< i+1; j++)
                SOP( twoD[i][j] + "  ");
            SOP ( );
        }
    }
}
```

Output

```
0
1  2
3  4  5
6  7  8  9
```

0,0			
1,0	1,1		
2,0	2,1	2,2	
3,0	3,1	3,2	3,3

the created array twoD.

Demonstration of a three-dimensional array:

```
class ThreeDMatrix
{
    PSVM (String args[])
    {
        int threeD [][][] = new int [3][4][5];
        int i, j, K;

        for(i=0; i<3; i++)
            for( j=0; j<4; j++)
                for(K=0; K<5; K++)
                    threeD[i][j][K] = i * j * K;

        for (i=0; i<3; i++)
        {
            for( j=0; j<4; j++)
            {
                for(K=0; K<5; K++)
                    sop (threeD[i][j][K] + " ");

                sop();
            }
            sop();
        }
    }
}
```

Output

```
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

0 0 0 0 0
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12

0 0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24
```

Arrays in Java is implemented as objects. There is an instance variable "length" present in class which holds the size of array.

```
class Length
{
    psvm ( String args[])
    {
        int a1[] = new int[10];
        int a2[] = { 3, 5, 7, 1, 8, -10};
        int a3[] = { 4, 3, 2, 1};
        sop(" Length of a1 is " + a1.length);   // 10
        sop (" Length of a2 is " + a2.length);  // 6
        sop (" Length of a3 is " + a3.length);  // 4
    }
}
```

Example of a stack class:

```
class stack
{
    private int stack[];
    private int tos;

    stack(int size)
    {   stack = new int[size];
        tos = -1;
    }
    void push(int item)
    {
        if (tos == stack.length-1)
            sop(" stack is full");
        else
            stack[++tos] = item;
    }
    int pop()
    {
        if (tos<0)
        {   sop(" stack overflow");
            return 0;
        }
        else return stack[tos--];
    }
}
```

```
class StackTest
{
    psvm( String args[])
    {
        Stack s1 = new Stack(5);
        Stack s2 = new stack(8);

        for(int i=0; i<5; i++)
            s1.push(i);
        for(int i=0; i<8; i++)
            s2.push(i);
        sop(" stack in s1 :");
        for(int i=0; i<5; i++)
            sop(s1.pop());
        sop(" stack in s2 :");
        for(int i=0; i<8; i++)
            sop(s2.pop());
    }
}
```

JAVA Strings

- JAVA implements strings as a object of type String.
- Every string you create is an object of type String. String constants are also String objects.

```
System.out.println ("This is a String");
```

- String are immutable; i.e once a String object is created, its content cannot be changed.
- If you need to change a string, you have to create ~~new~~ new one that contains the modifications.
- Java defines another class called StringBuffer, which allows strings to be altered.

Example:

```
String s1 = "this is a new String";
System.out.println (s1);
```

- '+' operator concatenate two strings.

```
String s2 = "I" + "like" + "Java.";
System.out.println (s2);
```

String class Profile:

Constructors:

(1) String(): This constructor is used for creating an empty string.

Example: `String s= new String();`

(2) String(String): This constructor is used for creating an object of String class with some String data.

```
String s = new String("JAVA");
sop(s); // JAVA
```

(3) String(char[]): This constructor is used for converting array of characters into string.

Example:

```
char ch[] = {'J', 'A', 'V', 'A'};
String s = new String(ch);
SOP(s); // JAVA
```

(4) String(char chars[], int startIndex, int numChars): This constructor allows to specify a subrange of character array as an initializer.

```
char chars[] = {'a', 'b', 'c', 'd', 'e', 'f'};
String s = new String(chars, 2, 3);
sop(s); // cde
```

(5) String(StringBuffer): This constructor is used for converting StringBuffer class object into String class object.

In otherwords it is converting mutable object into immutable object.

Example:

```
StringBuffer sb1 = new StringBuffer("JAVA");
String s1 = new String(sb1);
sop(s1); // JAVA.
```

Methods of String class :

① int length() :

This method is used for finding no. of characters present in String object.

Example : ①
```
String s[] = {"JAVA", "C++"};
sop("No of elements in S = "+s.length);//2
int noc = s[0].length();
sop(noc); // 4
```

② toString() :

String toString() :

If you want to represent any object as a string then toString method is used.

toString() returns the string representation of the object.

Example :

```
class Box
{
  double w;
  double h;
  double d;
Box(double x, double y
    double z)
{
  w = x;
  h = y;
  d = z;
}
public String toString()
{
  return "Dimensions are"
  w + " by " + d +
  " by " + h + ".";
}}
```

```
class toStringDemo
{
  PSVM(String args[])
  {
    Box b = new Box(10,20,30);
    String s = "Box b:" + b;
    sop(b); //
    sop(s); //
  }
}
```

③ char charAt(int):

This method is used for finding a character by passing a valid position of a String data.

Example:
```
String s = "JAVA";
char ch = s.charAt(2);
sop(ch); // V
```

④ void getChars(int sourceStart, int sourceEnd, char target, int targetStart):

Example:

This method is used to extract more than one char.

```
String s = "This is an Example";
int start = 8;
int end = 10;
char buf[] = new char[end-start];
s.getchars(start, end, buf, 0);
sop(buf); // an
```

⑤ char[] toCharArray():

This method is used for converting string data into array of characters.

Example:
```
String s = "JAVA";
char ch[] = s.toCharArray();
for(int i=0; i<ch.length; i++)
{   sopoint(ch[i]);
}
```

⑥ equals() and equalsIgnoreCase():

```
boolean equals(Object sto);
```

This method is used for equality of two strings. Here sto is string object being compared with invoking object.

To perform a comparision ignoring the case we use equalsIgnoreCase() method.

Example :

```
String s1 = "Hello";
String s2 = "Hello";
String s3 = "Good Bye";
String s4 = "HELLO";
sop(s1.equals(s2)); // true
sop(s1.equals(s3)); // false
sop(s1.equals(s4)); // false
sop(s1.equalsIgnoreCase()); // true
```

equals() versus == :

equals() method compares the characters inside a String object.

== operator compares two objects references to see whether they refere to the same instance.

Example :

```
String s1 = "Hello";
String s2 = new String(s1);
sop(s1.equals(s2)); // true
sop(s1 == s2); // false
```

⑧ String substring(int startIndex) :

This method is used for obtaining a part of the string from specified position to last.

Example :

```
String s = "Java Prog";
String ss = s.substring(2);
sop(ss); // va Prog
```

The second form of substring() allows you to specify both beginning and ending index of the substring.

```
String s = "JAVA PROG";
String ss = s.substring(2,6);
SOP(ss); // VA PR
```

(9) String concat(String str):

This method creates a new object appending the string to the end.

```
String s1 = "one";
String s2 = s1.concat("two");
sop(s2); // onetwo
```

(10) String trim():

This method is used for eliminating leading and trailing blank spaces but not the blank spaces within the String.

```
String s = "   JAVA program   ";
```

leading spaces — trailing spaces

```
SOP(s.length());
String s1 = s.trim();
SOP(s1.length());
sop(s1);
```

StringBuffer class profile:

Constructors:

StringBuffer defines these four constructors.

1. StringBuffer():
2. StringBuffer(String):
3. StringBuffer(char[]):
4. StringBuffer(~~StringBuffer~~ int size):

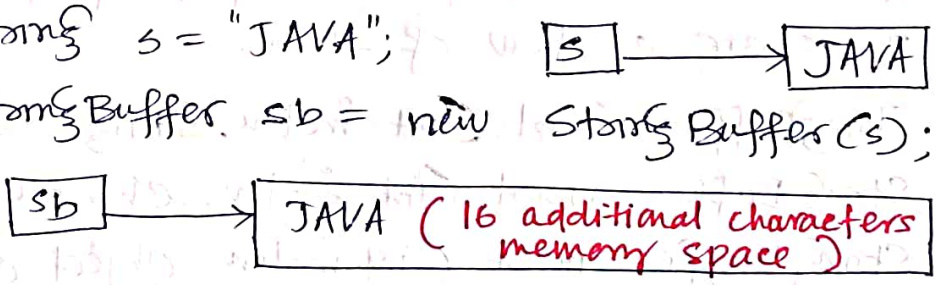
The default constructor reserves room for 16 characters without reallocation.

The second version accepts a String argument that initializes the StringBuffer object and reserves room for 16 more characters without reallocation.

The third constructor creates an object and contains the character contained in char[] and reserves room for 16 more characters.

The last version accepts an integer argument and sets the size of buffer.

```
String s = "JAVA";
StringBuffer sb = new StringBuffer(s);
```




```
SOP(sb); // JAVA
```

Methods of StringBuffer class:

1. int length() and int capacity():

Total length can be found by length() and allocated capacity can be found by capacity() method.

```
StringBuffer sb = new StringBuffer("hello");
SOP(sb.length()); // 5
SOP(sb.capacity()); // 21
```

② append() :

The append() method concatenates the string representation of any other type of data to the end of the invoking StringBuffer object.

This method is overloaded and has ~~three~~ many forms. Here is some of them.

```
StringBuffer append(String str);
StringBuffer append(int num);
StringBuffer append(Object obj);
```

```
String s;
int a = 42;
StringBuffer sb = new StringBuffer(40);
s = sb.append("a = ").append(a).append("!").
                                    toString();
sop(s); // a = 42!
```

③ insert() :

The insert() method inserts one string into another. It is a overloaded method and it has several forms. These are a few of them.

```
StringBuffer insert(int index, String str);
StringBuffer insert(int index, char ch);
StringBuffer insert(int index, Object obj);
```

```
StringBuffer sb = new StringBuffer("I Java!");
sb.insert(2, "like");
sop(sb); // I like Java!
```

④ reverse():

This method reverse the characters within a StringBuffer object.

```
StringBuffer s= new StringBuffer("abcdef");
s.reverse();
sop(s); // fedcba
```

⑤ delete() and deleteCharAt():

You can delete characters within a StringBuffer by using the methods delete() and deleteCharAt().

```
StringBuffer delete(int startIndex, int endIndex):
String Buffer delete( int loc):

StringBuffer sb= new StringBuffer("This is a test.");
sb.delete(4,7);
sop(sb); // This a test.
sb.deleteCharAt(0);
sop(sb); // his a test.
```

⑥ replace():

You can replace one set of characters with another set inside a StringBuffer object.
Its signature is:

```
StringBuffer replace(int startIndex, int endIndex, String sb):

StringBuffer sb= new StringBuffer("This is a test.");
sb.replace(5,7, "was");
sop(sb); // This was a test
```

= o =